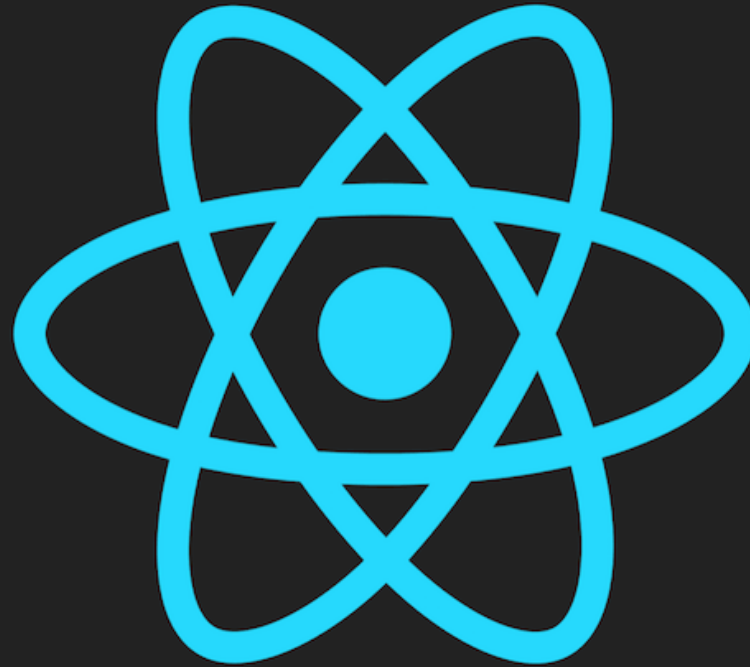




React Native

Conceitos Básicos

Componentes

- Podem ser criados através de **função** ou de **classe**;
- Retornam um elemento React;
- Através de Hooks API React é possível adicionar **State** a um componente de função.

Componente de Função

```
import React from 'react';
import {Text, View} from 'react-native';

const HelloWorldApp = () => {
  return (
    <View
      style={{
        flex: 1,
        justifyContent: 'center',
        alignItems: 'center',
      }}>
      <Text>Hello, world!</Text>
    </View>
  );
};

export default HelloWorldApp;
```

Componente de Classe

```
import React, {Component} from 'react';
import {Text, View} from 'react-native';

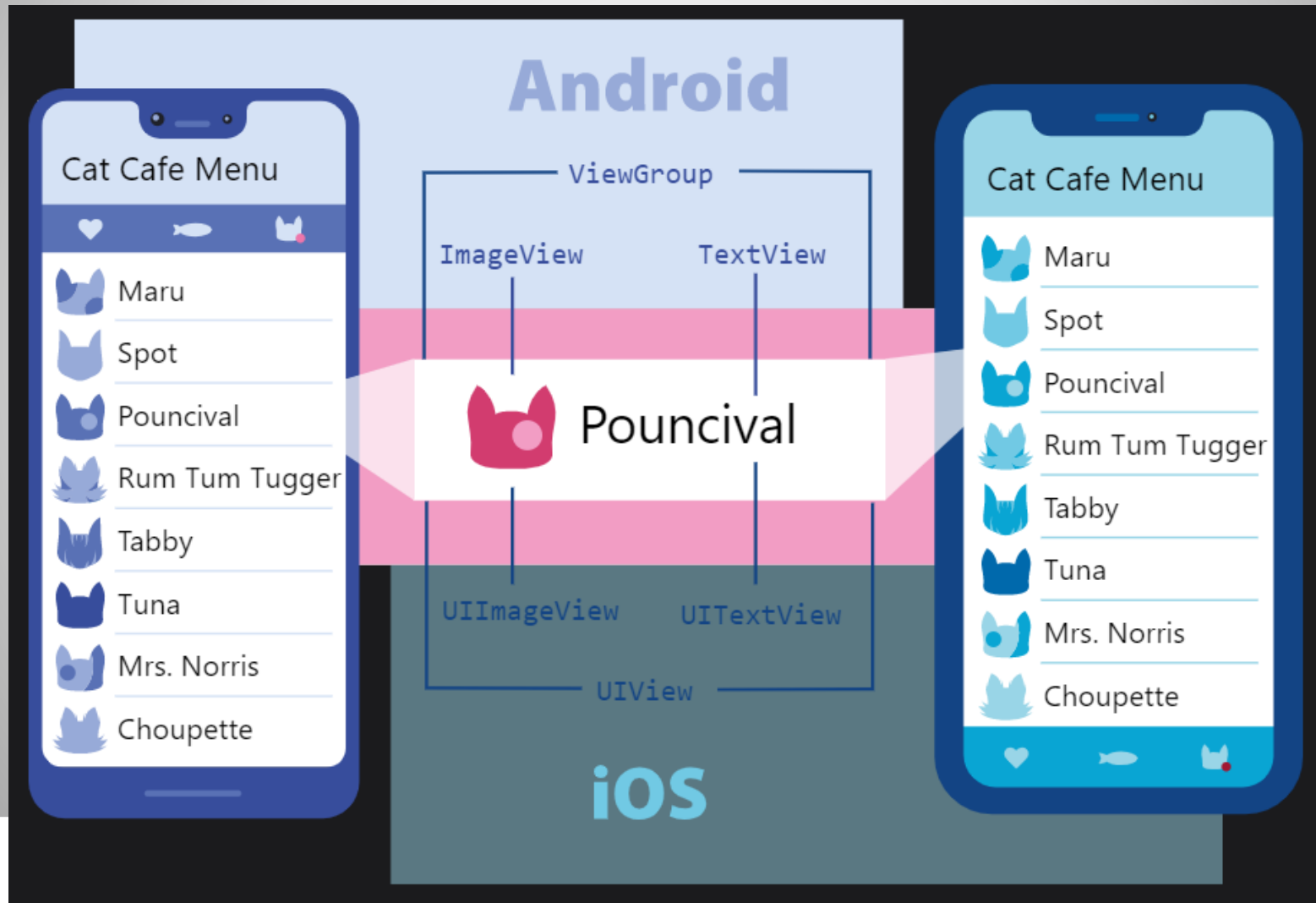
class HelloWorldApp extends Component {
  render() {
    return (
      <View
        style={{
          flex: 1,
          justifyContent: 'center',
          alignItems: 'center',
        }}>
        <Text>Hello, world!</Text>
      </View>
    );
  }
}

export default HelloWorldApp;
```

Views

- Em Android e iOS, uma *view* é o bloco básico de construção de Interface de Usuário (UI);
- Um pequeno elemento retangular na tela usado para exibir texto, imagens, etc.;
- Todo elemento visual é uma espécie de *view*;
- Algumas *views* podem conter outras *views*.

Views



Componentes Nativos

- Em React Native é possível invocar views de Android e iOS com JavaScript usando componentes React;
- Criação dos componentes correspondentes nessas plataformas em tempo de execução;
- São os componentes de plataformas;
- Possível criar componentes Android ou iOS customizados.

Core Components

REACT NATIVE UI COMPONENT	ANDROID VIEW	IOS VIEW	WEB ANALOG	DESCRIPTION
<code><View></code>	<code><ViewGroup></code>	<code><UIView></code>	A non-scrolling <code><div></code>	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
<code><Text></code>	<code><TextView></code>	<code><UITextView></code>	<code><p></code>	Displays, styles, and nests strings of text and even handles touch events
<code><Image></code>	<code><ImageView></code>	<code><UIImageView></code>	<code></code>	Displays different types of images
<code><ScrollView></code>	<code><ScrollView></code>	<code><UIScrollView></code>	<code><div></code>	A generic scrolling container that can contain multiple components and views
<code><TextInput></code>	<code><EditText></code>	<code><UITextField></code>	<code><input type="text"></code>	Allows the user to enter text

Core Components

Hello World ⓘ ↗

^ Expo

```
import React from 'react';
import {View, Text, Image, ScrollView, TextInput} from 'react-native';

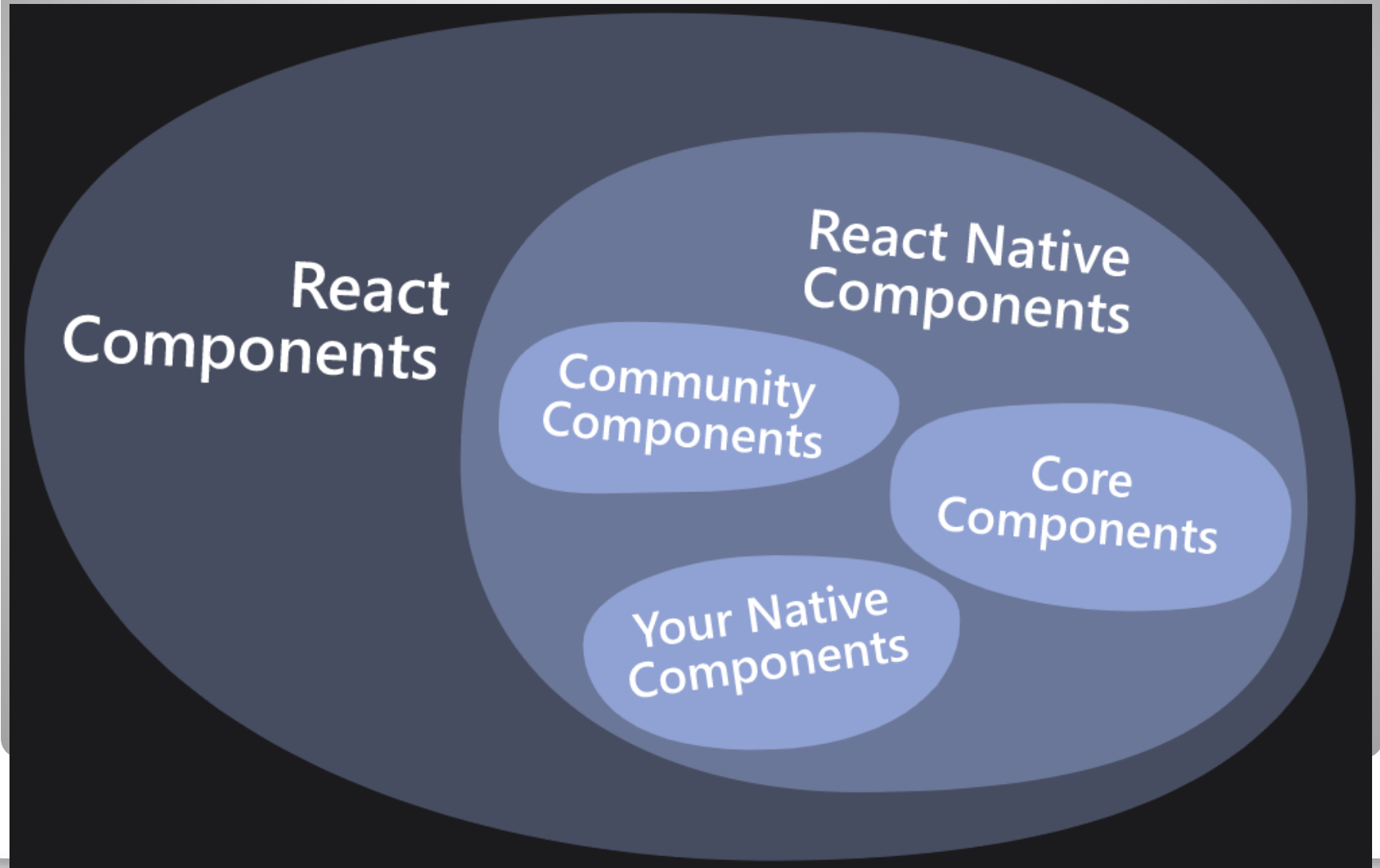
const App = () => {
  return (
    <ScrollView>
      <Text>Some text</Text>
      <View>
        <Text>Some more text</Text>
        <Image
          source={{
            uri: 'https://reactnative.dev/docs/assets/p_cat2.png',
          }}
          style={{width: 200, height: 200}}
        />
      </View>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1,
        }}
        defaultValue="You can type in me"
      />
    </ScrollView>
  );
};
```

Some text
Some more text



You can type in me

Core Components



Fundamentos React

JSX

- React e React Native utilizam JSX;
- Permite escrever elementos dentro de JavaScript;
- Permite incorporar expressões JS dentro de elementos React.

JSX

```
import React from 'react';
import {Text} from 'react-native';

const getFullName = (firstName, secondName, thirdName) => {
  return firstName + ' ' + secondName + ' ' + thirdName;
};

const Cat = () => {
  return <Text>Hello, I am {getFullName('Rum', 'Tum', 'Tugger')}!</Text>;
};

export default Cat;
```

Componentes Customizados

- É possível aninhar componentes nativos dentro de outros componentes.

```
import React from 'react';
import {Text, TextInput, View} from 'react-native';

const Cat = () => {
  return (
    <View>
      <Text>Hello, I am...</Text>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1,
        }}
        defaultValue="Name me!"
      />
    </View>
  );
};

export default Cat;
```

Componentes Customizados

- Esses componentes podem ser renderizados múltiplas vezes.

```
import React from 'react';
import {Text, View} from 'react-native';

const Cat = () => {
  return (
    <View>
      <Text>I am also a cat!</Text>
    </View>
  );
};

const Cafe = () => {
  return (
    <View>
      <Text>Welcome!</Text>
      <Cat />
      <Cat />
      <Cat />
    </View>
  );
};

export default Cafe;
```


Props

- Permite passar parâmetros para os elementos criados.

```
import React from 'react';
import {Text, View} from 'react-native';

const Cat = props => {
  return (
    <View>
      <Text>Hello, I am {props.name}!</Text>
    </View>
  );
};

const Cafe = () => {
  return (
    <View>
      <Cat name="Maru" />
      <Cat name="Jellylorum" />
      <Cat name="Spot" />
    </View>
  );
};

export default Cafe;
```

Props

Props ⓘ ↗

^ Expo

```
import React from 'react';
import {Text, View, Image} from 'react-native';

const CatApp = () => {
  return (
    <View>
      <Image
        source={{
          uri: 'https://reactnative.dev/docs/assets/p_cat1.png',
        }}
        style={{width: 200, height: 200}}
      />
      <Text>Hello, I am your cat!</Text>
    </View>
  );
};

export default CatApp;
```



Hello, I am your cat!

State

- Mantém o estado interno de um componente;
- Útil para gerenciar dados que mudam com o tempo ou com a interação do usuário;
- Pode ser implementado em componentes de função chamando *useState* Hook.

State

State and Class Components

```
import React, {Component} from 'react';
import {Button, Text, View} from 'react-native';

class Cat extends Component {
  state = {isHungry: true};

  render() {
    return (
      <View>
        <Text>
          I am {this.props.name}, and I am
          {this.state.isHungry ? ' hungry' : ' full'}!
        </Text>
        <Button
          onPress={() => {
            this.setState({isHungry: false});
          }}
          disabled={!this.state.isHungry}
          title={
            this.state.isHungry ? 'Pour me some milk, please!' : 'Thank
you!'
          }
        />
      </View>
    );
  }
}
```

State – Componente de Classe

```
class Cafe extends Component {  
  render() {  
    return (  
      <>  
        <Cat name="Munkustrap" />  
        <Cat name="Spot" />  
      </>  
    );  
  }  
}  
  
export default Cafe;
```

State – Componente de Função

```
import React, {useState} from 'react';
import {Button, Text, View} from 'react-native';

const Cat = props => {
  const [isHungry, setIsHungry] = useState(true);

  return (
    <View>
      <Text>
        I am {props.name}, and I am {isHungry ? 'hungry' : 'full'}!
      </Text>
      <Button
        onPress={() => {
          setIsHungry(false);
        }}
        disabled={!isHungry}
        title={isHungry ? 'Pour me some milk, please!' : 'Thank you!'}
      />
    </View>
  );
};
```

State – Componente de Função

```
const Cafe = () => {  
  return (  
    <>  
      <Cat name="Munkustrap" />  
      <Cat name="Spot" />  
    </>  
  );  
};  
  
export default Cafe;
```

Gerenciando Listas

ScrollView

- Container de rolagem genérico que pode conter múltiplos componentes e views;
- A rolagem pode ser **horizontal ou vertical** (propriedade *horizontal*);
- Renderiza todos os elementos, mesmo os que não estão sendo vistos.

ScrollView

```
import React from 'react';
import {Image, ScrollView, Text} from 'react-native';

const logo = {
  uri: 'https://reactnative.dev/img/tiny_logo.png',
  width: 64,
  height: 64,
};

const App = () => (
  <ScrollView>
    <Text style={{fontSize: 96}}>What's the best</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 96}}>Framework around?</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{fontSize: 80}}>React Native</Text>
  </ScrollView>
);
```

List Views

- O React Native provê dois componentes para listas: **FlatList** e o **SectionList**;

FlatList

- Recomendado para longas listas cuja **quantidade de itens muda** com o tempo;
- Renderiza apenas os elementos visíveis;
- Requer duas propriedades:
 - **data**: a lista
 - **renderItem**: recebe um item como entrada e retorna um componente formatado

FlatList

```
const FlatListBasics = () => {  
  return (  
    <View style={styles.container}>  
      <FlatList  
        data={[  
          {key: 'Devin'},  
          {key: 'Dan'},  
          {key: 'Dominic'},  
          {key: 'Jackson'},  
          {key: 'James'},  
          {key: 'Joel'},  
          {key: 'John'},  
          {key: 'Jillian'},  
          {key: 'Jimmy'},  
          {key: 'Julie'},  
        ]}  
        renderItem={({item}) => <Text style={styles.item}>{item.key}</Text>  
      />  
    </View>  
  );  
};
```

SectionList

- Para conjunto de dados divididos em seções lógicas.

```
const SectionListBasics = () => {
  return (
    <View style={styles.container}>
      <SectionList
        sections={[
          {title: 'D', data: ['Devin', 'Dan', 'Dominic']},
          {
            title: 'J',
            data: [
              'Jackson',
              'James',
              'Jillian',
              'Jimmy',
              'Joel',
              'John',
              'Julie',
            ],
          },
        ]}
        renderItem={({item}) => <Text style={styles.item}>{item}</Text>}
        renderSectionHeader={({section}) => (
          <Text style={styles.sectionHeader}>{section.title}</Text>
        )}
        keyExtractor={item => `basicListEntry-${item}`}
      />
    </View>
  );
};
```